ModulAsso : document d’architecture

Version du 8 octobre 2026. Ce document est la référence technique du projet. Il explique ce que fait chaque partie, pourquoi elle existe et comment elles se parlent. Il est écrit pour que chaque membre de l’équipe, développeur ou non, comprenne l’ensemble sans avoir lu le code.

Documents liés : `modele-donnees` (les 41 tables) et les maquettes (Canvas Claude Design).

---

1. Le projet en deux minutes

ModulAsso est une application web pour les associations : disponibilités des bénévoles, événements, groupes, échéances à ne pas rater, stock et matériel. Elle est gratuite, pensée pour le téléphone (PWA installable) et modulaire : chaque association active seulement les modules dont elle a besoin.

Trois types d’utilisateurs :

Qui	Ce qu’il fait	Où
Bénévole	Donne ses disponibilités, voit ses tâches et ses groupes	Application publique (mobile surtout)
Dirigeant	Gère la structure, les membres, les modules, les invitations	Application publique (PC et mobile)
Super admin (nous)	Gère la plateforme : rôles, catalogue de modules, statistiques, supervision	Application admin, accessible uniquement par VPN

Trois principes qui guident tous les choix techniques :

1. **Sécurité d’abord.** Aucun secret exposé, API invisible depuis Internet, aucune fuite tolérée.
2. **Simple à maintenir à deux.** Une seule API découpée par domaine, pas de microservices.
3. **Données minimales.** On ne garde que le nécessaire (RGPD).

---

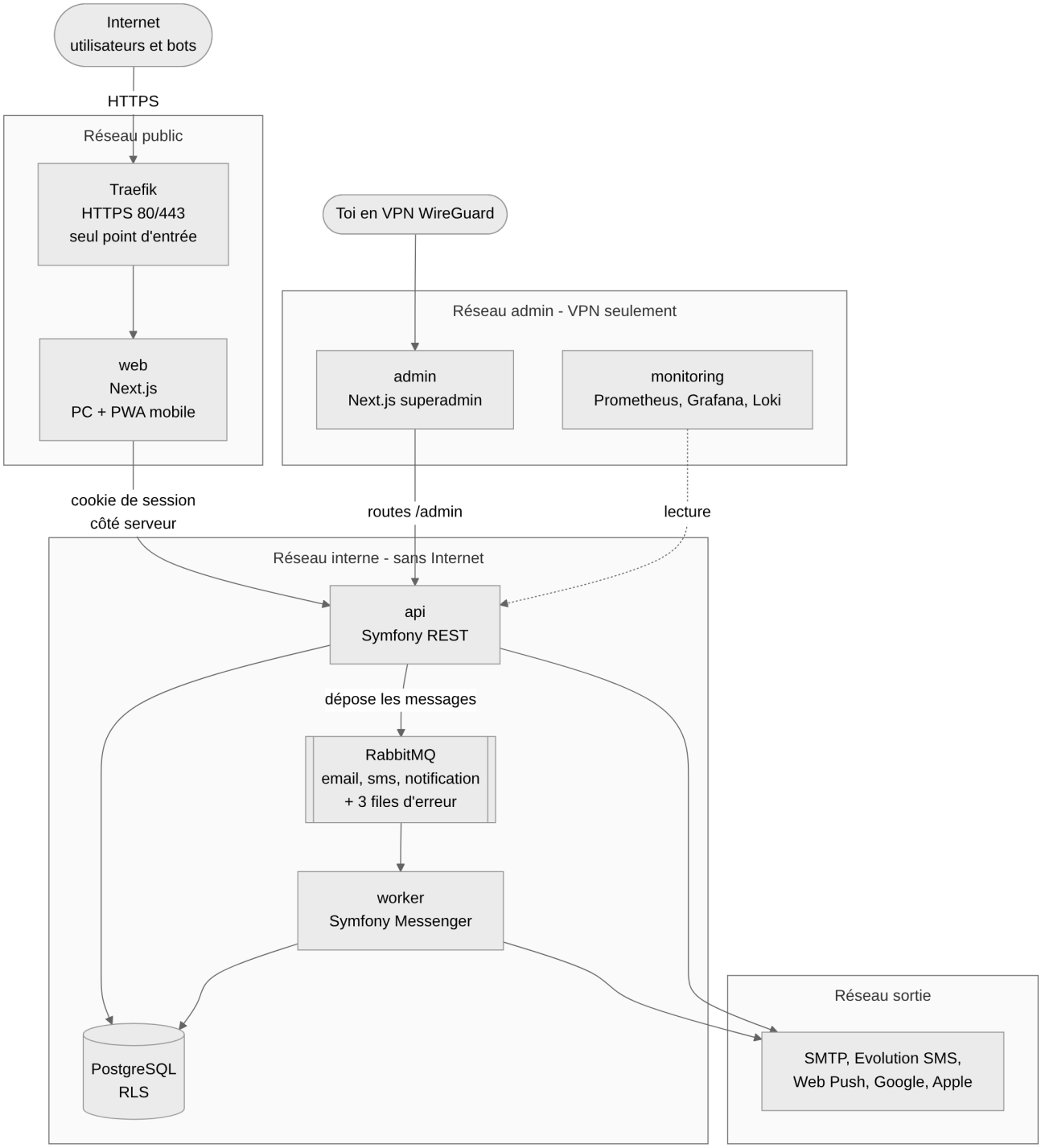
2. Glossaire

Terme	Sens
Structure	Une association cliente (ou club, collectif…). Aussi appelée « tenant » côté technique.
Membre	Le lien entre un utilisateur et une structure, avec un rôle.
Module	Un bloc fonctionnel activable : planning, échéances, stock, matériel.
Socle	Ce qui est commun à toutes les structures : comptes, rôles, invitations, équipes, notifications.

Terme	Sens
RLS	Sécurité par ligne de PostgreSQL : la base elle-même refuse de montrer les données d’une autre structure.
PWA	Site installable sur le téléphone, avec notifications push et lecture hors ligne.
Worker	Programme qui tourne en arrière-plan et traite les tâches (envoi de mails, imports…).
File (queue)	Boîte où l’on dépose des messages à traiter plus tard.
Dead letter / file d’erreur	File où atterrissent les messages qui ont échoué plusieurs fois.
VPN	Tunnel privé : sans lui, l’application admin n’est pas joignable.
Docker secret	Fichier secret monté dans un conteneur, jamais dans une variable d’environnement.

3. Vue d’ensemble

3.1 Schéma général



À retenir :

- **Un seul point d’entrée public : Traefik.** Il ne route que vers `web`.
- **Le navigateur ne parle jamais à l’API.** Seul le serveur Next.js (`web`) l’appelle.
- **L’API, la base et RabbitMQ n’ont aucun port publié.** Ils sont sur un réseau Docker interne coupé d’Internet.
- **L’admin n’existe pas côté public.** Elle écoute uniquement sur l’IP du VPN.

3.2 Les services

Service	Techno	Rôle	Réseaux	Port publié
traefik	Traefik	HTTPS, Let’s Encrypt, filtre les requêtes, bloque les fichiers cachés	public	80, 443
web	Next.js	Interface publique, bureau et mobile (PWA)	public, interne	aucun (derrière Traefik)
admin	Next.js	Console super admin, monitoring, statistiques	admin	aucun (IP du VPN seulement)
api	Symfony	API REST, toutes les règles métier	interne, sortie, admin	aucun
worker	Symfony (même image que api)	Envoie mails, SMS, push ; traite imports et purges	interne, sortie	aucun
db	PostgreSQL	Base unique, 41 tables, RLS	interne	aucun
rabbitmq	RabbitMQ	Files d’envoi (mail, SMS, notification)	interne	aucun
monitoring	Prometheus, Grafana, Loki, Alertmanager	Supervision technique	admin	aucun
mailpit	Mailpit	Faux SMTP, développement uniquement	dev	local

3.3 Les réseaux Docker

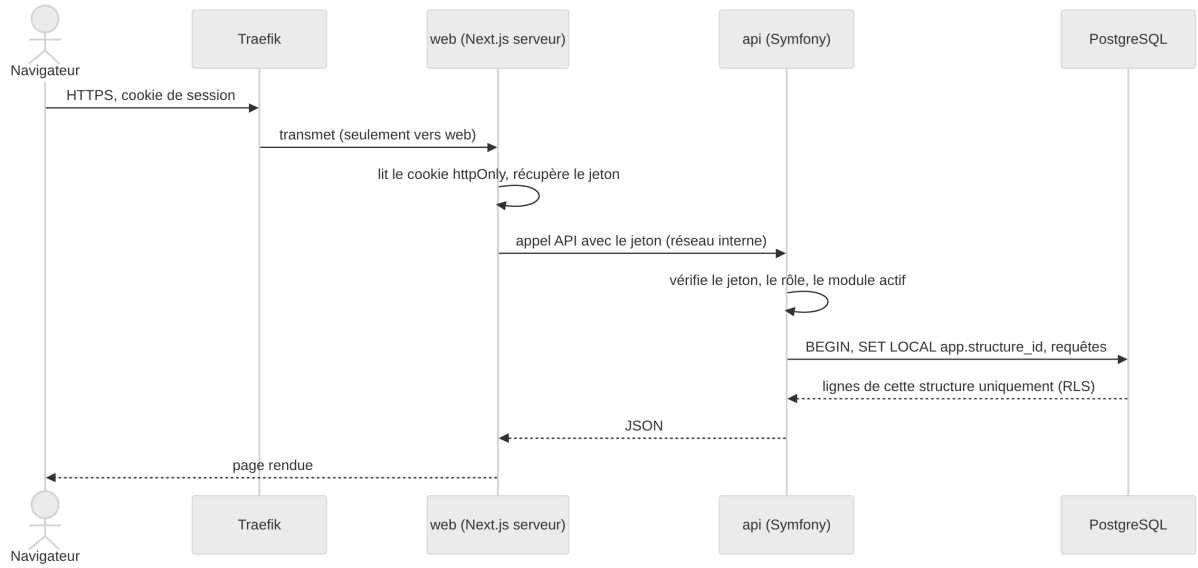
Réseau	Contient	Accès Internet	Pourquoi
public	traefik, web	entrant via Traefik	Seul réseau exposé
interne	web (côté serveur), api, worker, db, rabbitmq	non (`internal: true`)	Isole les données de tout ce qui vient d’Internet
sortie	api, worker	sortant seulement	Envoyer mails, push, SMS, appeler Google et Apple
admin	admin, api, monitoring	non	Console réservée au VPN

La base et RabbitMQ sont **uniquement** sur le réseau interne : même si quelqu’un compromet `web`, il ne peut atteindre la base qu’à travers l’API.

---

4. Comment une requête circule

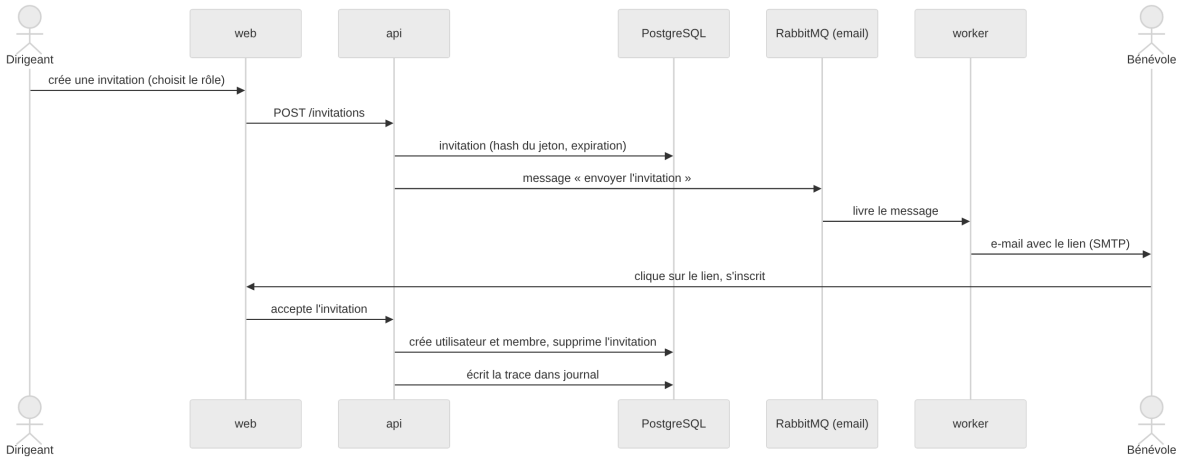
4.1 Une requête utilisateur normale



Chaque étape protège quelque chose :

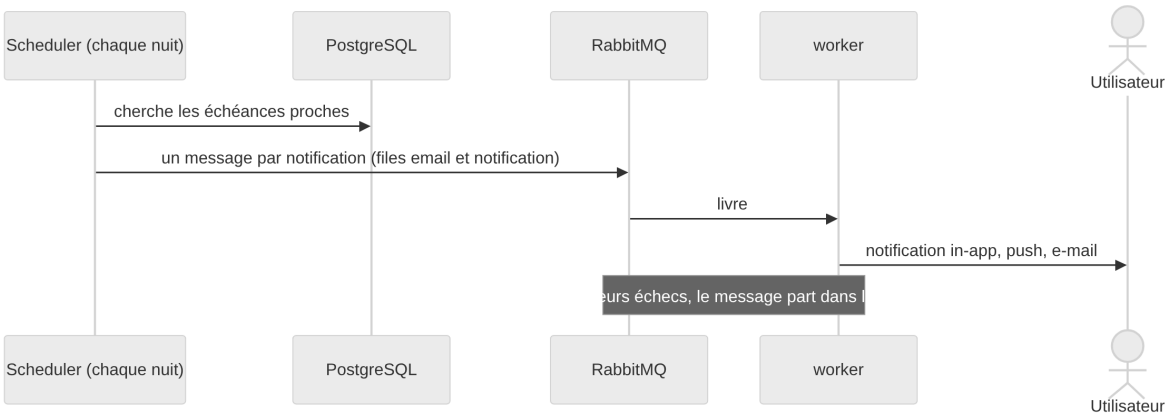
1. Le **cookie httpOnly** est illisible par le JavaScript du navigateur : un script malveillant ne peut pas voler le jeton.
2. L'API vérifie le **jeton** (qui es-tu), le **rôle** (as-tu le droit) et le **module** (la structure l'a-t-elle activé).
3. La **structure courante** est posée en base avec `SET LOCAL app.structure_id`. La RLS filtre alors toutes les requêtes : même un bug dans le code ne peut pas lire les données d'une autre structure.

4.2 L'invitation d'un bénévole



Le lien contient le jeton en clair, la base ne garde que son empreinte (SHA-256). La ligne d'invitation est supprimée dès l'inscription, la trace reste dans le journal.

4.3 Un rappel d’échéance



---

5. Le front public (`apps/web`)

5.1 Rôle

Une seule application Next.js qui affiche l’interface bureau et l’interface mobile. Elle appelle l’API côté serveur uniquement.

5.2 Organisation

```
apps/web/
├── src/
│   ├── app/                        # routes uniquement, pages courtes qui assemblent des composants
│   │   ├── (auth)/                 # connexion, inscription, invitation
│   │   ├── (bureau)/               # mise en page PC
│   │   ├── (mobile)/               # mise en page mobile (PWA)
│   │   └── api/                    # route handlers : session, proxy vers l'API
│   │
│   ├── components/                 # composants partagés par 2 features ou plus
│   │   ├── ui/                     # Button, Input, Select, Modal, Card, Badge, Toast, Tabs, Avatar
│   │   ├── forms/                  # FormField, DatePicker, PasswordInput, FileDropzone
│   │   ├── data/                   # Table, List, EmptyState, Pagination, SearchBar, StatusPill
│   │   ├── layout/                 # Sidebar, TopBar, BottomNav, PageHeader
│   │   ├── calendar/               # grille de jours, sélecteur de statut
│   │   ├── dashboard/              # grille glisser-déposer, WidgetFrame
│   │   └── feedback/               # Loader, ErrorBoundary, ConfirmDialog
│   │
│   ├── features/                   # un dossier par module
│   │   ├── socle/                  # membres, équipes, groupes, import, profil
│   │   ├── planning/
│   │   ├── echeances/
│   │   ├── stock/
│   │   └── materiel/
│   │       ├── components/         # composants propres à la feature
│   │       ├── hooks/
│   │       ├── actions.ts          # server actions
│   │       └── types.ts
│   │
│   ├── hooks/                      # useMediaQuery, useDebounce, useSession
│   ├── lib/
│   │   └── config.ts               # seul endroit qui lit la configuration et les secrets
│   └── middleware.ts               # session, redirection bureau/mobile, en-têtes
│
├── public/                         # icônes PWA, manifeste
├── sw.ts                           # service worker (Serwist) : cache hors ligne, push
├── next.config.ts                  # CSP, HSTS, en-têtes de sécurité
├── .env.example                    # noms des variables, valeurs factices
├── .dockerignore                   # exclut .env*, .git, tests
└── Dockerfile
```

5.3 Règles

- **Les pages `app/` sont courtes** : elles assemblent des composants, sans logique.
- **Où va un composant ?** Utilisé par une seule feature : dans `features/<x>/components/`. Utilisé par deux features ou plus : il monte dans `components/`.
- **Un composant = un dossier** (`Button/Button.tsx`, `Button.test.tsx`, `index.ts`). Un fichier ne dépasse pas environ 150 lignes.
- **Bureau et mobile partagent tout sauf `components/layout/`.** Seule la mise en page change.
- **Choix bureau ou mobile** : redirection automatique selon la taille d'écran, choix manuel possible (la table `vue_dashboard` garde une vue par appareil).
- **Un module désactivé disparaît de la navigation.**
- **Aucune URL d'API écrite à la main** : on utilise le client généré `packages/api-client`.
- **Aucune variable `NEXT_PUBLIC_*` ne contient un secret** : elles sont embarquées dans le code envoyé au navigateur.

5.4 PWA

Installable sur le téléphone, lecture hors ligne de l'agenda, des tâches et des disponibilités, abonnement aux notifications push. Sur iPhone, les push fonctionnent une fois la PWA ajoutée à

l’écran d’accueil.

---

6. Le front admin (`apps/admin`)

6.1 Rôle

Console du super admin, **application séparée** du front public : son propre conteneur, sa propre image, son propre réseau. Aucun code de l’admin n’est présent dans le front public, donc rien à scanner pour un bot.

6.2 Organisation

```
apps/admin/
├── src/
│   ├── app/
│   │   ├── (auth)/               # connexion
│   │   ├── (console)/            # pages protégées
│   │   │   ├── monitoring/       # état des services, files RabbitMQ et erreurs, Scheduler, disque,
dernière sauvegarde
│   │   │   ├── statistiques/     # structures, utilisateurs, modules, activité, notifications
(agrégats)
│   │   │   ├── structures/       # liste, détail, suspension
│   │   │   ├── utilisateurs/     # recherche, désactivation, anonymisation RGPD
│   │   │   ├── roles/            # rôles et permissions (seul lieu de création)
│   │   │   ├── modules/          # catalogue, disponibilité, dépendances, versions
│   │   │   ├── files/            # files d'erreur, relance manuelle
│   │   │   ├── imports/          # suivi des imports (résumés uniquement)
│   │   │   └── journal/          # historique des actions
│   │   └── api/                  # session uniquement
│   ├── components/               # propres à l'admin (ui, data, charts, layout, feedback)
│   ├── features/                 # monitoring, statistiques, structures, utilisateurs, roles, modules,
files, imports, journal
│   ├── lib/                      # session, config
│   └── middleware.ts             # session obligatoire, en-têtes, blocage hors VPN
├── .env.example, .dockerignore
├── next.config.ts                # CSP stricte, aucune PWA
└── Dockerfile
```

6.3 Règles de sécurité

- Écoute **uniquement sur l’IP WireGuard**, jamais derrière le Traefik public, aucun nom de domaine public.
- Appelle uniquement les routes `/admin` de l’API, qui refusent toute requête hors réseau admin ou sans `est_super_admin`.
- Session courte (30 minutes d’inactivité), cookie httpOnly, limitation des tentatives de connexion.
- Chaque action sensible (suspension, anonymisation, relance de file) écrit une ligne dans `journal`.
- Aucune donnée personnelle en clair : pas de téléphone, statistiques en agrégats uniquement.
- Composants propres : aucune dépendance avec le front public.

6.4 Monitoring et statistiques

Monitoring (technique) : état de chaque service, taille des files RabbitMQ et des files d’erreur, dernier passage du Scheduler, espace disque, dernière sauvegarde réussie. Alimenté par la route `/admin/health`.

Statistiques (usage de la plateforme), alimentées par `/admin/stats` :

- Structures : total, par type, nouvelles par semaine, actives sur 30 jours.
- Utilisateurs : inscrits, nouveaux par semaine, connectés sur 7 et 30 jours, répartition par rôle.
- Taille : membres par structure, invitations en attente ou expirées.
- Modules : nombre de structures par module activé.
- Activité : événements, tâches, échéances, imports réussis ou en erreur.
- Notifications : envoyées par canal, taux d'échec.

Les chiffres sont **calculés à la volée** depuis les tables existantes (aucune table de statistiques), avec un cache de quelques minutes côté API. Les courbes de croissance viennent des dates de création (`cree_le`, `rejoint_le`, `active_le`). Limite connue : une suppression ou anonymisation n'est pas reflétée dans l'historique des courbes. Les « connectés sur 7 et 30 jours » viennent du `journal` (une ligne `connexion`, sans IP ni appareil), donc limités à sa durée de conservation. Un rôle Postgres dédié `modulasso_stats` lit seulement des vues agrégées.

---

7. L'API (`apps/api`)

7.1 Rôle

Une seule API Symfony (REST, JSON) qui porte toutes les règles métier. Elle n'est jamais appelée par un navigateur : seulement par `web`, `admin` et le `worker`.

7.2 Organisation

```
apps/api/
├── src/
│   ├── Controller/              # routes uniquement
│   │   ├── Auth/                # AuthController, AuthGoogleController, AuthAppleController
│   │   ├── Utilisateur/         # UtilisateurController, UtilisateurRgpdController
│   │   ├── Structure/           # StructureController, MembreController, EquipeController,
SousEquipeController
│   │   ├── Invitation/          # InvitationController, InvitationGroupeController
│   │   ├── Planning/            # EvenementController, ParticipationController, GroupeController,
DisponibiliteController, TacheController
│   │   ├── Stock/               # ArticleController, LotController, MouvementStockController
│   │   ├── Materiel/            # MaterielController, PretMaterielController
│   │   ├── Echeances/           # EcheanceController
│   │   ├── Import/              # ImportController
│   │   ├── Notification/        # NotificationController, AbonnementPushController
│   │   ├── Dashboard/           # DashboardController
│   │   └── Admin/               # AdminHealthController, AdminStatsController, ...
│   │
│   ├── Service/                 # toute la logique métier, mêmes dossiers que Controller
│   │   ├── Auth/ Utilisateur/ Structure/ Invitation/ Planning/ Stock/ Materiel/
│   │   ├── Echeances/ Import/ Notification/ Dashboard/ Admin/
│   │   └── Shared/              # Tenant, Chiffrement, Journal, activation des modules, base CRUD
│   │
│   ├── Entity/                  # une entité par table
│   ├── Repository/              # requêtes Doctrine uniquement
│   ├── Dto/                     # Request/ (entrées validées) et Response/ (sorties)
│   ├── Security/                # voters, authenticators, provider
│   ├── Attribute/               # #[RequiresModule]
│   ├── EventListener/           # structure courante (SET LOCAL), module actif, zone admin
│   ├── Message/                 # messages Messenger (email, sms, notification, import, purge)
│   ├── MessageHandler/          # un handler par message
│   ├── Command/                 # tâches du Scheduler : échéances, purges RGPD
│   └── Exception/
├── config/                      # packages, routes, messenger.yaml
├── migrations/                  # SQL : tables, CHECK, index, RLS
├── tests/                       # mêmes dossiers que src
├── .env.example                 # développement uniquement
├── .dockerignore
└── Dockerfile
```

7.3 Règles : qui fait quoi

Couche	Fait	Ne fait jamais
Controller	Reçoit la requête, valide l'entrée (DTO), appelle un service, renvoie le JSON	Aucune règle métier, aucune requête SQL
Service	Porte la règle métier, appelle repositories et autres services	Ne connaît ni la requête HTTP ni le JSON
Repository	Requêtes Doctrine	Aucune règle métier

Pourquoi cette séparation : la même règle est appelée depuis plusieurs endroits. Créer une invitation se fait depuis le bouton du dirigeant **et** depuis l'import de membres (une invitation par ligne). La règle vit dans le service, les deux l'appellent, un bug de sécurité se corrige une seule fois.

Un fichier par ressource et par contrôleur (`Controller/Stock/ArticleController.php`, `Service/Stock/ArticleService.php`). Les ressources simples (lieu, équipe, article, matériel…) héritent d'une petite classe de base dans `Service/Shared/` pour ne pas réécrire le CRUD.

7.4 Sécurité de l’API

- **Authentification** : l’API émet un jeton court (15 minutes) et un jeton de rafraîchissement. Connexion par e-mail et mot de passe (argon2id), Google ou Apple. Le serveur Next.js garde les jetons dans un cookie httpOnly.
- **Droits** : des voters lisent `role_permission`. Les rôles sont globaux, créés seulement par le super admin, un seul rôle par membre.
- **Modules** : l’attribut `#[RequiresModule('stock')]` sur un contrôleur refuse la requête (403) si le module n’est pas actif pour la structure.
- **Isolation des structures** : voir section 8 (RLS).
- **Routes** `/admin` : refusées hors réseau admin ou sans `est_super_admin`.
- **Données sensibles** : téléphone chiffré par l’application (libsodium), mots de passe en argon2id, jetons stockés en SHA-256.
- **Logs** : jamais d’e-mail, de téléphone ni de jeton.

---

8. La base de données

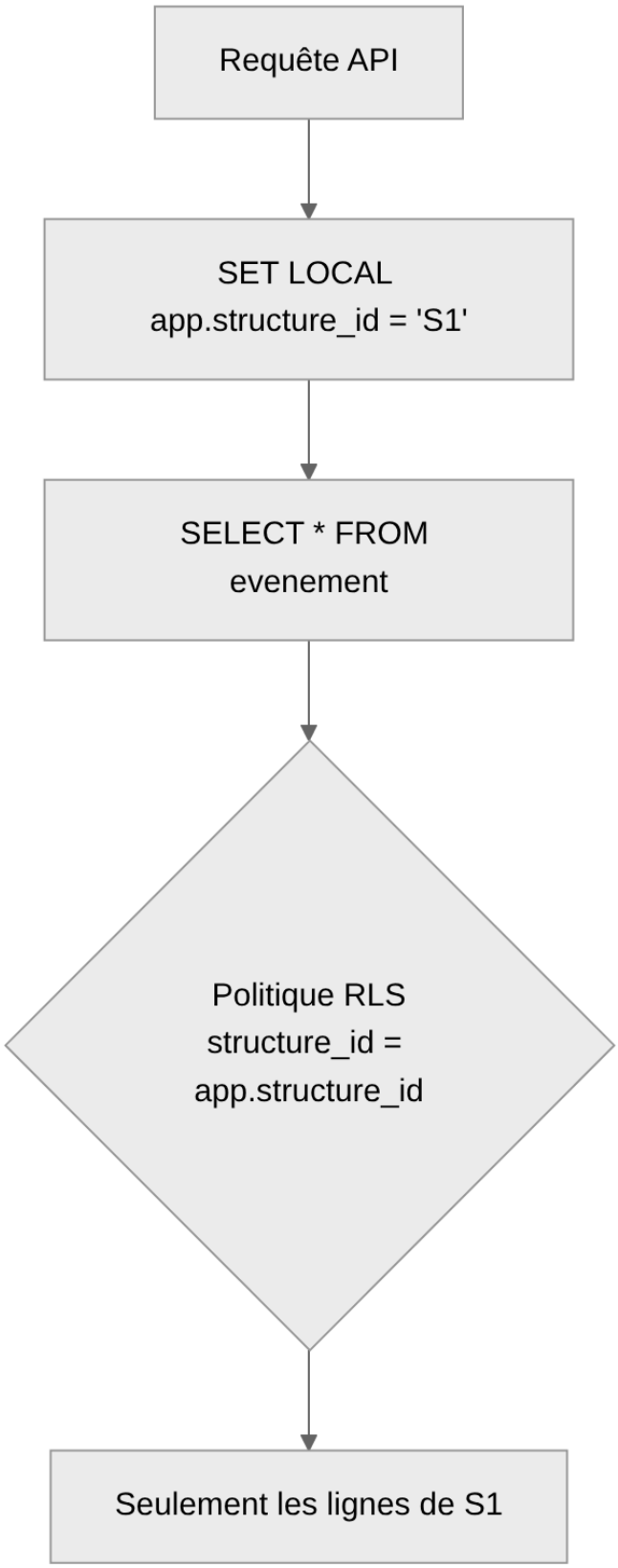
8.1 Principe

Une seule base PostgreSQL, 41 tables (détail dans le document `modele-donnees`). Chaque table d’entité porte `structure_id`. La sécurité par ligne (RLS) filtre dessus.

Sections de tables : comptes et connexion, rôles et permissions, structures et membres, équipes, événements, groupes, planning et tâches, échéances, notifications, stock, matériel, modules, tableau de bord, import, journal.

Conventions : identifiants en `uuid` (le journal en `bigint`), `timestamptz` pour un moment précis, `varchar` avec `CHECK` pour les listes de valeurs, `jsonb` pour les réglages, clés étrangères en `_id`, un index sur chaque `structure_id`.

8.2 Comment la RLS isole les structures



Sans `SET LOCAL`, la politique ne renvoie rien. Le rôle applicatif n’est pas propriétaire des tables, il ne peut donc pas désactiver la RLS.

8.3 Les rôles PostgreSQL

Rôle	Utilisé par	Droits
`modulasso_owner`	Migrations uniquement	Crée et modifie le schéma
`modulasso_app`	api et worker	Lecture et écriture des données, soumis à la RLS, aucun droit sur le schéma
`modulasso_stats`	Route `/admin/stats`	Lecture seule sur des vues agrégées

8.4 RGPD

Sujet	Règle
Données collectées	Le strict nécessaire. E-mail obligatoire, nom, prénom, téléphone, photo facultatifs
Téléphone	Chiffré par l'application
Mots de passe, jetons	argon2id, SHA-256
Suppression de compte	Anonymisation de `utilisateur` (`supprime_le` rempli)
Journal	Jamais de donnée personnelle dans `details`. Conservation proposée : 12 mois
Purges automatiques	Notifications lues anciennes, invitations expirées, données brutes d'import après validation
Droits de l'utilisateur	Export et suppression de ses données depuis son profil
Logs applicatifs	Sans e-mail, téléphone ni jeton
Sauvegardes	Chiffrées, durée de rétention limitée (elles contiennent des données personnelles)

8.5 Sauvegardes

`pg_dump` quotidien chiffré, dans un volume séparé de celui de la base. Pour l'instant sur le même serveur : elles ne protègent donc pas d'une perte du serveur, une copie externe sera à prévoir.

8.6 Volumes

Base, RabbitMQ et sauvegardes sont des volumes nommés. Supprimer ou recréer un conteneur (`docker compose down`, mise à jour d'image) ne les touche pas. **Seul `docker compose down -v` les supprime : ne jamais l'utiliser en production.**

---

9. Les files de messages

9.1 Deux systèmes, deux usages

Système	Pour quoi	Pourquoi
RabbitMQ	Tout ce qui part vers l'extérieur : e-mail, SMS, notification	Ajouter un canal ne change rien au reste
File dans PostgreSQL (Messenger)	Tâches internes : imports, purges RGPD, échéances	Elles écrivent déjà dans la base, la tâche vit avec ses données, pas de message perdu entre les deux

Si le volume grossit, déplacer les tâches internes vers RabbitMQ est un simple changement de configuration Messenger.

9.2 Les files RabbitMQ

File	Usage	File d'erreur
`email`	Invitations, rappels, réinitialisations	`email-error`
`sms` (plus tard, via Evolution)	SMS	`sms-error`
`notification`	Push et notifications dans l'app	`notification-error`

Un message qui échoue plusieurs fois (par exemple SMTP en panne) part dans la file d'erreur, où il attend une relance manuelle depuis la console admin (page `files/`). Les files sont durables et les messages persistants : les envois en attente survivent à un redémarrage.

9.3 Imports

Un import ne bloque jamais la page : l'API répond « import en cours », le worker travaille en arrière-plan.

1. Téléversement du fichier (Excel, CSV, Google Sheets).
2. Contrôle de chaque ligne dans `import_ligne`.
3. Aperçu avec les erreurs ligne par ligne.
4. Validation par le dirigeant.
5. Création en une seule transaction : tout passe ou rien ne passe.
6. Suppression du fichier et des données brutes, seul le résumé reste.

L'import de membres envoie une invitation par e-mail à chaque personne au lieu de créer des comptes.

---

10. Éléments externes

Élément	Usage	Appelé par
Serveur SMTP	Envoi de mails	worker
Evolution	SMS (plus tard)	worker
Web Push (VAPID)	Alertes sur téléphone	worker

Élément	Usage	Appelé par
Google OAuth	Connexion	api
Google Sheets API	Import	api
Apple Sign In	Connexion	api
Let’s Encrypt	Certificats HTTPS	Traefik
Nom de domaine	Accès HTTPS au front	à prendre

Seuls `api` et `worker` ont accès à Internet en sortie.

---

11. Sécurité : vue d’ensemble

11.1 Secrets

- **Aucun fichier `.env` en production.**
- Chaque secret est un petit fichier (`db_password`, `jwt_key`…) dans un dossier du serveur, hors dépôt et hors de tout dossier servi, `chmod 600` pour root.
- Docker le monte en lecture seule dans le conteneur concerné sous `/run/secrets/`.
- Symfony le lit via `%env(file:...)%`, Next.js via `lib/config.ts`.
- Rien dans les images, dans Git, ni dans les variables d’environnement des conteneurs (donc absent de `docker inspect` et des pages d’erreur).
- Les réglages non secrets (noms d’hôtes, ports) restent dans le compose.
- En développement : `.env.local` ignoré par Git et par `.dockerignore`, plus un `.env.example` versionné avec des valeurs factices.
- Sur un nouveau serveur, les secrets sont recréés à la main depuis le gestionnaire de mots de passe. Le dossier `infra/secrets/` ne contient qu’un README listant les fichiers à créer.
- En CI et en pre-commit : gitleaks bloque tout commit contenant un secret.

11.2 Conteneurs

Utilisateur non-root, système de fichiers en lecture seule, `cap_drop: ALL`, `no-new-privileges`, limites CPU et mémoire, images épinglées par digest, scan Trivy en CI. Le socket Docker n’est monté nulle part (Traefik est configuré par fichier).

11.3 Exposition web

Traefik refuse les fichiers cachés (`.env`, `.git`…), ajoute les en-têtes de sécurité (HSTS, CSP), limite le débit, et CrowdSec bloque les bots. Next.js ajoute sa propre CSP.

11.4 Serveur

La configuration du serveur (pare-feu, SSH, VPN WireGuard) est déjà faite et sécurisée. Elle n’est pas détaillée ici.

11.5 Tableau récapitulatif des menaces

Menace	Protection
Bot qui cherche un `.env` ou `.git`	Aucun `.env` en prod, `.dockerignore`, blocage Traefik
Attaque directe sur l'API	Aucun port publié, réseau interne, appelée seulement par web
Vol de jeton par script (XSS)	Cookie httpOnly, CSP stricte
Lecture des données d'une autre structure	RLS, rôle applicatif non propriétaire
Accès à la console admin	VPN obligatoire, IP du VPN seulement, mot de passe, session courte, route `/admin` filtrée
Fuite d'une image Docker	Aucun secret dedans
Évasion d'un conteneur	Non-root, lecture seule, `cap_drop: ALL`
Fuite de données personnelles dans les logs	Filtrage des logs, journal sans donnée personnelle

---

12. Dépôt et infrastructure

12.1 Structure du dépôt

```
modulasso/
├── apps/
│   ├── web/            # front public (Next.js), Dockerfile inclus
│   ├── admin/          # superadmin (Next.js, VPN), Dockerfile inclus
│   └── api/            # Symfony, Dockerfile inclus (réutilisé par le worker)
├── packages/
│   └── api-client/     # client TypeScript généré depuis l'OpenAPI, sans logique
├── infra/
│   ├── compose.yml     # services, réseaux, volumes, secrets
│   ├── compose.dev.yml # surcharges dev : mailpit, ports locaux
│   ├── traefik/        # config, routes (web uniquement), en-têtes, limitation de débit
│   ├── rabbitmq/       # définition des 6 files, utilisateurs, relances
│   ├── postgres/       # rôles (owner, app, stats), extensions
│   ├── backup/         # pg_dump quotidien chiffré, rétention
│   ├── monitoring/     # prometheus, grafana, loki, alertmanager
│   └── secrets/        # README seul
└── docs/               # ce document, modèle de données, maquettes
```

Les migrations vivent dans `apps/api/migrations/` : la version du schéma suit le code qui l'utilise.

12.2 Environnements

	Développement	Production
Lancement	`docker compose up` (compose.yml + compose.dev.yml)	compose.yml
Mail	Mailpit	SMTP réel
Secrets	`.env.local`	Docker secrets
HTTPS	non	Traefik + Let's Encrypt
Services en plus	mailpit	traefik, monitoring

12.3 Intégration continue

À chaque push : lint, tests de l'API (base PostgreSQL réelle, RLS incluse), tests du front, gitleaks, scan Trivy, build des images.

---

13. Conventions de travail

- **Git** : branches par fonctionnalité, relecture obligatoire avant fusion, aucun secret commité.
- **Nommage** : tables, colonnes et routes en français comme dans le modèle de données ; code des composants et fonctions en anglais ou français de façon cohérente dans un même fichier.
- **Un fichier = une responsabilité** : un contrôleur par ressource, un service par ressource, un composant par dossier.
- **Tests** : chaque service a ses tests, chaque contrôleur au moins un test d'accès (droit, module actif, isolation entre structures).
- **Avancement** : étapes de deux semaines, un module fini et testé vaut mieux que trois à moitié faits.

---

14. Points encore ouverts

Sujet	État
Durée de conservation du `journal`	12 mois proposés, à confirmer
Double authentification (TOTP) sur l'admin	Question posée, pas de réponse
Monitoring : pile complète (Prometheus, Grafana, Loki) ou page native dans l'admin d'abord	Question posée, pas de réponse
Admin dans le même dépôt Git ou dépôt privé séparé	Question posée, pas de réponse
Disponibilités : journée ou créneaux horaires	À trancher (impacte la table `disponibilite`)
Politiques RLS des tables de liaison sans `structure_id`	À rédiger (héritage par jointure avec le parent)
Hébergement de production et nom de domaine	Domaine à prendre ; superadmin sur le même serveur pour l'instant, déplaçable plus tard
Copie externe des sauvegardes	À prévoir plus tard
Serveur superadmin séparé	Possible plus tard

---

15. Par où commencer

1. `infra/` minimal : compose avec db, api, web, rabbitmq, réseaux et secrets de dev.
2. Migration SQL des 41 tables, contraintes CHECK, index, RLS, rôles PostgreSQL.
3. Socle : comptes, connexion, rôles, structures, invitations.
4. Premier écran de bout en bout (connexion puis tableau de bord), pour valider toute la chaîne web, api, base.
5. Ensuite les modules, un par un : planning, échéances, notifications, import, stock, matériel.